

The Physics of a Cricket Ball in Flight

A trajectory model and control algorithm for a twin-wheel bowling machine

Covers: gravity, speed-dependent aerodynamic drag, spin-induced Magnus force, seam-induced swing, numerical trajectory integration, and the wheel-speed control mapping for a physical machine.

12 September 2026

1. Introduction and scope

This document builds a physics-based model of a cricket ball's flight from release to pitching, and turns that model into two algorithms: a **forward simulator** (given a release speed, angle, spin and seam presentation, predict the full trajectory) and a **machine control mapping** (given the delivery you want, compute the wheel speeds a twin-wheel bowling machine needs to produce it).

The model combines five forces: gravity, aerodynamic drag, the spin-induced Magnus force, a seam-induced swing force, and the extra drag that same seam presentation costs. The first two apply to any ball in flight; the third is what makes spin bowling turn and drift; the fourth and fifth are two sides of the same distinct mechanism behind swing bowling — a sideways component and a companion drag component, resolved from one seam-generated force (Sections 3.4-3.5). All five are summed into one equation of motion and integrated numerically, because — unlike a textbook projectile problem — the forces here depend on the ball's own velocity, so there is no closed-form solution for the trajectory.

Honesty note: the drag, lift and swing coefficients used throughout (C_d , C_l , C_s) are empirical approximations built to reproduce the correct qualitative behaviour (a drag crisis near bowling Reynolds numbers, a saturating Magnus lift, a swing peak near a ~20-25° seam angle). They are not derived from wind-tunnel data for a specific ball. Section 8 explains exactly how to replace them with numbers measured from your own machine and ball, which is what turns this from an illustrative model into a predictive one.

2. Coordinate system and state representation

The ball's state at any instant is six numbers: a position and a velocity, each with three components.

$$\mathbf{s} = (x, y, z, v_x, v_y, v_z)$$

Axes (right-handed): **x** runs down the pitch, from the machine toward the batting crease; **y** is lateral, positive toward the off side; **z** is vertical, positive up, with $z = 0$ at the ground. This choice is arbitrary but must be kept consistent everywhere — every force below is computed in this same frame.

The algorithm never solves for position as a function of time algebraically. Instead, it defines a function that computes acceleration from the current state, and steps the state forward in small time increments (Section 4). That one design decision is what allows every velocity-dependent force below to be included at all.

3. Forces acting on the ball in flight

3.1 Gravity

Constant, and the only force here that does not depend on velocity:

$$\vec{F}_g = (0, 0, -mg)$$

m is the ball's mass (≈ 0.160 kg for a men's ball) and $g = 9.81$ m/s².

3.2 Aerodynamic drag

Drag opposes the ball's motion through the air, with a magnitude that grows with the square of speed:

$$\vec{F}_{drag} = -\frac{1}{2} \rho C_d A |\vec{v}| \vec{v}$$

ρ is air density, A is the ball's cross-sectional area (πr^2), and $|\vec{v}| \cdot \vec{v}$ is speed multiplied into the velocity vector — this reproduces the v^2 scaling in magnitude while keeping the force pointing exactly opposite to whichever way the ball is currently travelling, not a fixed axis. C_d is the drag coefficient, and it is not constant — it depends on the Reynolds number:

$$Re = \frac{\rho |\vec{v}| D}{\mu}$$

D is the ball's diameter and μ is air's dynamic viscosity. Re compares how much the airflow is dominated by momentum (inertia) versus internal friction (viscosity) — low Re means the air flows smoothly around the ball in layers; high Re means it breaks up into a turbulent wake behind it. The drag coefficient is modelled as a smooth transition between two regimes:

$$C_d(Re) = C_{d, sup} + (C_{d, sub} - C_{d, sup}) \cdot \frac{1}{1 + e^{(Re - Re_{crit})/w}}$$

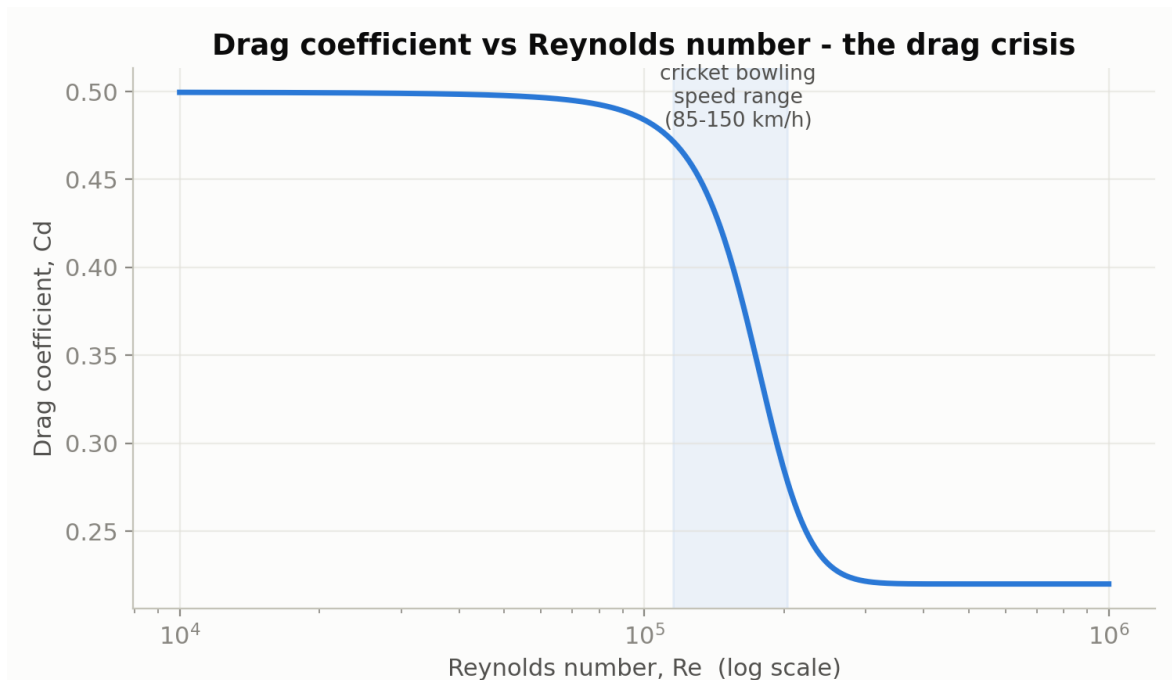


Figure 1. Modelled drag coefficient vs Reynolds number. Cricket bowling speeds (85-150 km/h) sit almost exactly on top of the transition — small changes in pace move the ball across a region where drag itself is changing fastest.

Why this matters for a bowling machine specifically: because bowling speed sits right on this transition, drag is not a smooth function of machine speed setting near typical operating speeds — expect a measurably

non-linear relationship between wheel RPM and delivered pace once you calibrate against a radar gun, rather than the simple linear one the wheel-speed equations in Section 7 assume before efficiency losses are accounted for.

3.3 Magnus force (spin)

A spinning ball drags a thin layer of air around with it by friction. That speeds up the airflow on one side and slows it on the other; by Bernoulli's principle, the faster side has lower pressure, producing a net sideways force:

$$\vec{F}_{Magnus} = \frac{1}{2} \rho C_l(S_p) A |\vec{v}|^2 (\hat{\omega} \times \hat{v})$$

The cross product of the spin-axis direction and the velocity direction is what turns "spinning" into a specifically *sideways* force, perpendicular to both the spin axis and the direction of travel. C_l is not constant either — it depends on the dimensionless spin parameter:

$$S_p = \frac{r \omega}{|\vec{v}|}$$

the ratio of the ball surface's spin speed to its speed through the air. It is modelled as a saturating curve:

$$C_l(S_p) = \frac{S_p}{a + S_p/C_{l,max}}$$

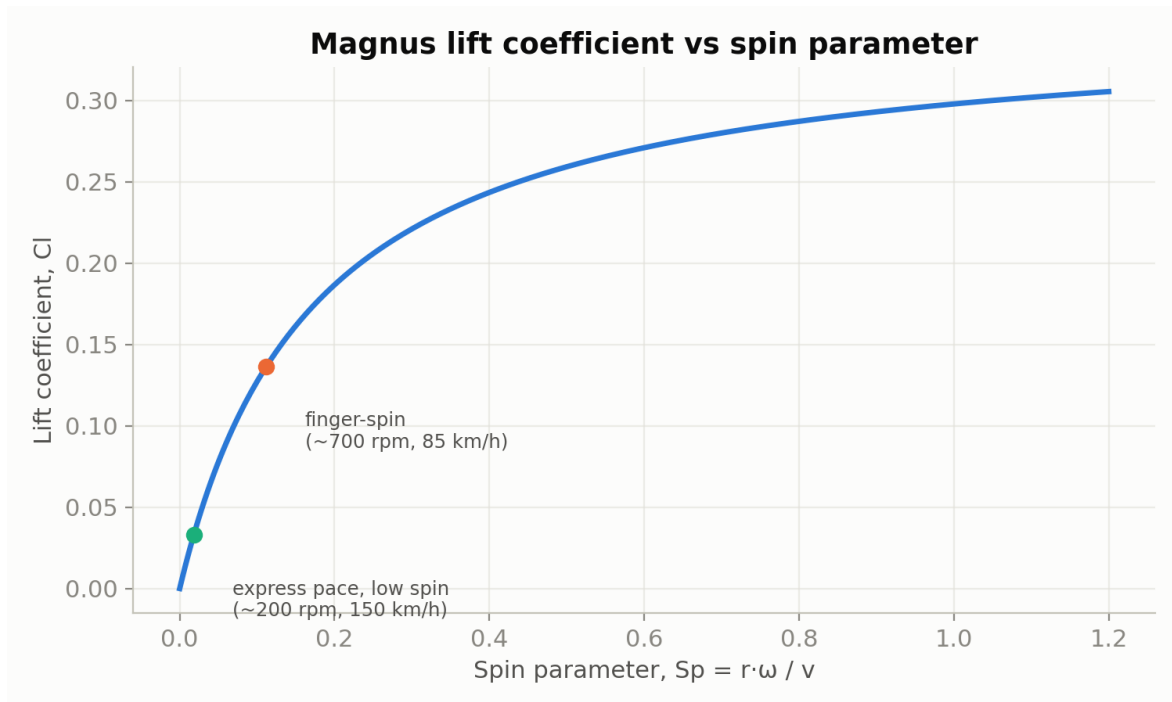


Figure 2. Magnus lift coefficient vs spin parameter. Finger-spin bowling operates at a much higher spin parameter than a fast, low-spin delivery, hence the very different Magnus force each produces even though air density and ball size are identical.

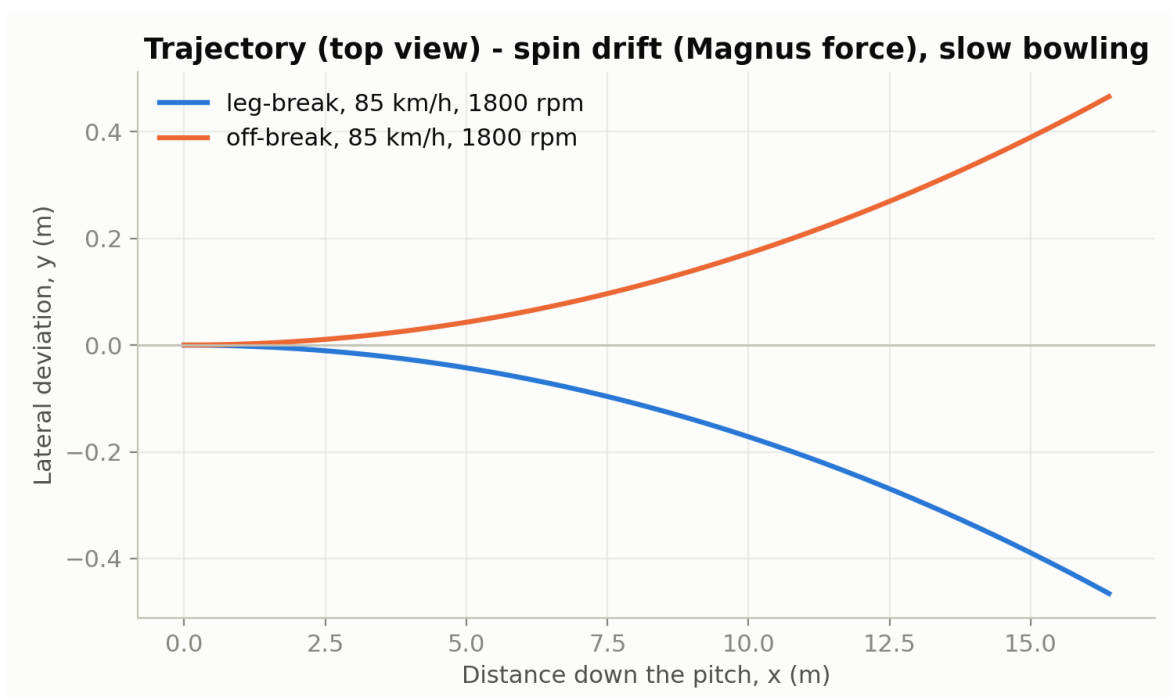


Figure 3. Simulated top-down drift for a leg-break and an off-break at the same pace and spin rate, opposite spin axes — the Magnus force reverses direction with the spin axis, as expected.

3.4 Swing force (seam)

Swing is a different mechanism from Magnus, and it is specifically a cricket-ball phenomenon: rather than fast overall spin, a pace bowler presents a stable, slowly-rotating seam at an angle to the direction of travel. The seam trips the boundary layer into turbulence earlier on one side of the ball than the other, so the two sides separate from the airflow at different points, producing a roughly constant-direction sideways force for the whole flight.

Concretely, that angle θ is just how far the seam is tilted away from pointing straight down the pitch, viewed from above:

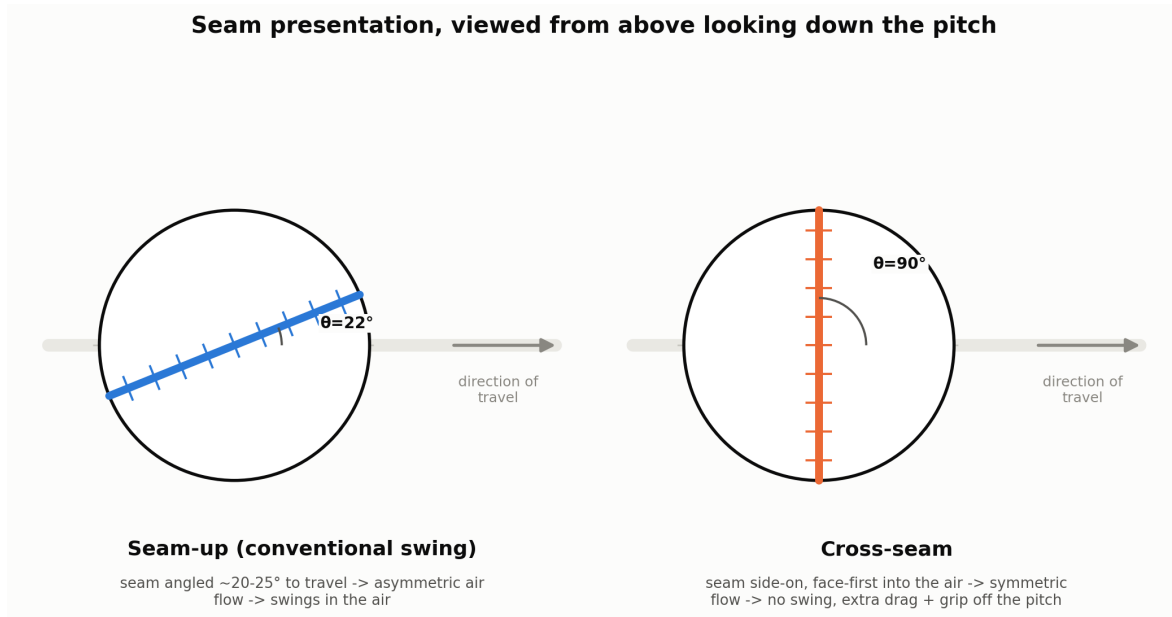


Figure 4. Seam orientation, viewed from above looking down the pitch. Left: a conventional seam-up delivery, seam tilted ~20-25° off straight — enough asymmetry to trip the boundary layer unevenly and swing. Right: cross-seam, seam rotated a full 90° so it faces the airflow symmetrically — no swing, just extra drag and a raised ridge for the pitch to grip (Section 3.5).

With that picture in mind, the resulting sideways force is:

$$\vec{F}_{swing} = \frac{1}{2} \rho C_s A |\vec{v}|^2 \hat{n}$$

Rather than fitting that lateral coefficient directly, it helps to think of the seam as generating a single aerodynamic side-force, which then has to be *resolved* into the direction that actually matters. Geometry says the swing (perpendicular-to-travel) component of that force should carry a $\cos(\theta)$ projection: as the seam rotates away from being broadside to the airflow, less of the asymmetry projects sideways and more of it projects along the direction of travel instead. Combined with the force itself growing from zero frontal exposure at $\theta = 0^\circ$, that gives:

$$C_s(\theta) = A_0 \sin \theta \cos^n \theta$$

which is zero at 0° (no asymmetry to project) and at 90° (fully side-on — everything projects into drag, not swing). A plain $\sin(\theta) \cdot \cos(\theta)$ would peak at 45° ; the exponent n (default 6) folds in the empirically-observed fact that the seam's own boundary-layer disruption saturates well before that, which is what pulls the combined peak down to the classically-quoted "ideal" seam angle of $\sim 22^\circ$ seen in Figure 5 below.

3.5 The companion effect: extra drag from an angled seam

The $\cos(\theta)$ resolution above has a natural counterpart: whatever fraction of the seam's side-force does *not* project into swing does not simply vanish — geometrically it projects along the direction of travel instead, as extra drag on top of Section 3.2's Reynolds-based value:

$$\vec{F}_{seam\ drag} = -\frac{1}{2} \rho C_{ds} A |\vec{v}| \vec{v}$$

This is modelled as its own curve rather than a strict second projection of the same swing force, because it reflects a genuinely different mechanism at the extreme end: simple frontal blockage, how much of the raised seam's profile faces the oncoming air, which keeps growing all the way out to a seam fully side-on:

$$C_{ds}(\theta) = C_{ds, max} \sin^2 \theta$$

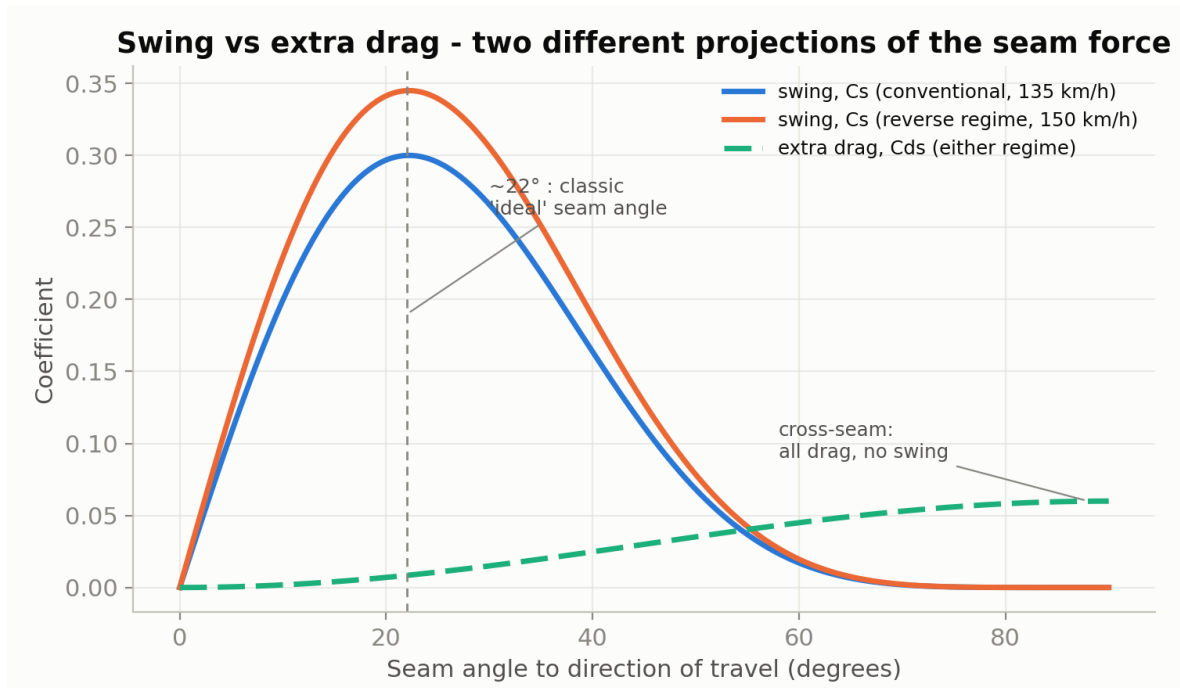


Figure 5. Swing coefficient (two projections, since a high-pace delivery switches to the reverse-swing regime past the pace threshold) alongside the companion extra-drag coefficient. They are deliberately not mirror images: swing peaks and fades by 90°, while extra drag keeps rising — exactly the "cross-seam" ball bowlers use on purpose for grip and bounce, at the cost of pace and with no swing at all.

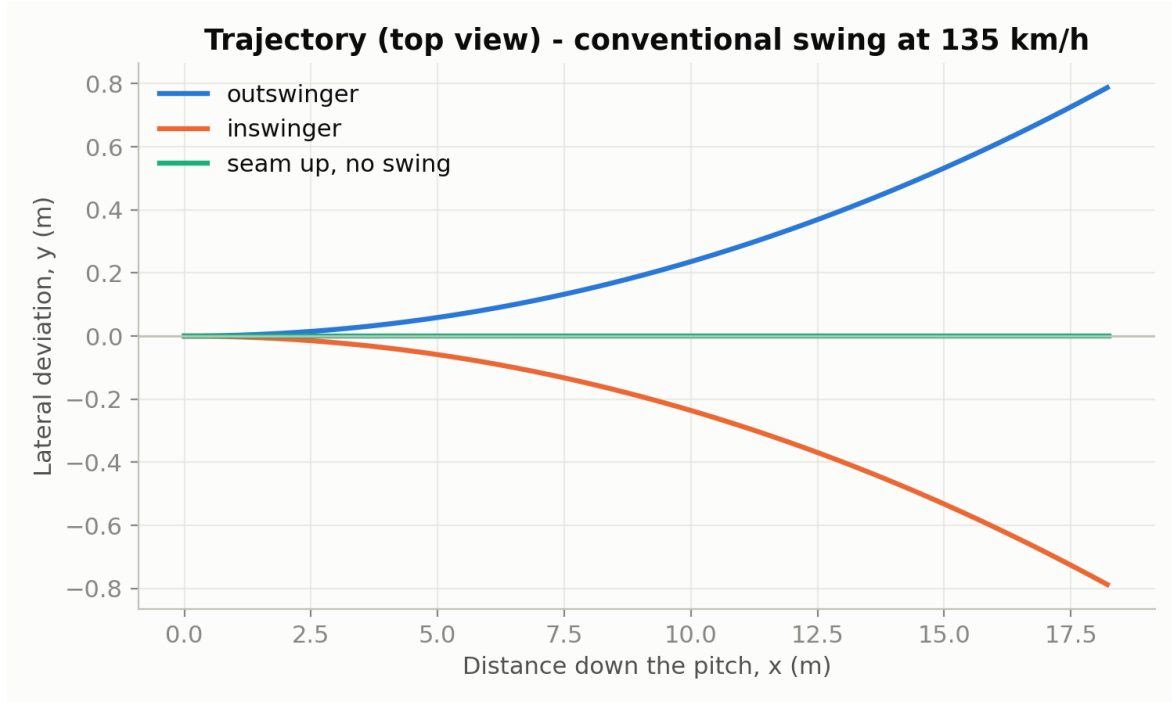


Figure 6. Simulated top-down swing for an outswinger, an inswinger, and a seam-up ball with no swing, all released at the same pace and angle — only the seam presentation changes.

Reverse swing (roughly above 140 km/h, when the shiny side effectively leads) is still an active research topic; the toggle in this model is a deliberately simplified stand-in, not a first-principles prediction. Treat any reverse-swing output as illustrative, and calibrate it against your own machine and ball condition if precision matters.

4. Equation of motion and numerical integration

Summing all five forces and dividing by mass gives the ball's total acceleration at any instant:

$$m \frac{d\vec{v}}{dt} = \vec{F}_g + \vec{F}_{drag} + \vec{F}_{Magnus} + \vec{F}_{swing} + \vec{F}_{seam\ drag}$$

Because this depends on the ball's own velocity (through drag, Magnus and swing), there is no algebraic formula for position as a function of time. Instead the state is marched forward numerically. The simplest version of that step (explicit Euler) is:

$$\vec{v}_{t+\Delta t} = \vec{v}_t + \vec{a}_t \Delta t \quad \vec{x}_{t+\Delta t} = \vec{x}_t + \vec{v}_t \Delta t$$

In practice, plain Euler steps accumulate error quickly for a fast-changing force like drag, so the reference implementation in Section 6 uses an adaptive Runge-Kutta integrator (RK45, via SciPy's `solve_ivp`) instead — same underlying idea, just recomputing acceleration at several sub-points within each step for much better accuracy, and shrinking the step automatically wherever the trajectory is changing fastest. Integration stops the instant `z` returns to 0 (an event condition on the solver), which is how the delivery's flight time and pitching point are obtained.

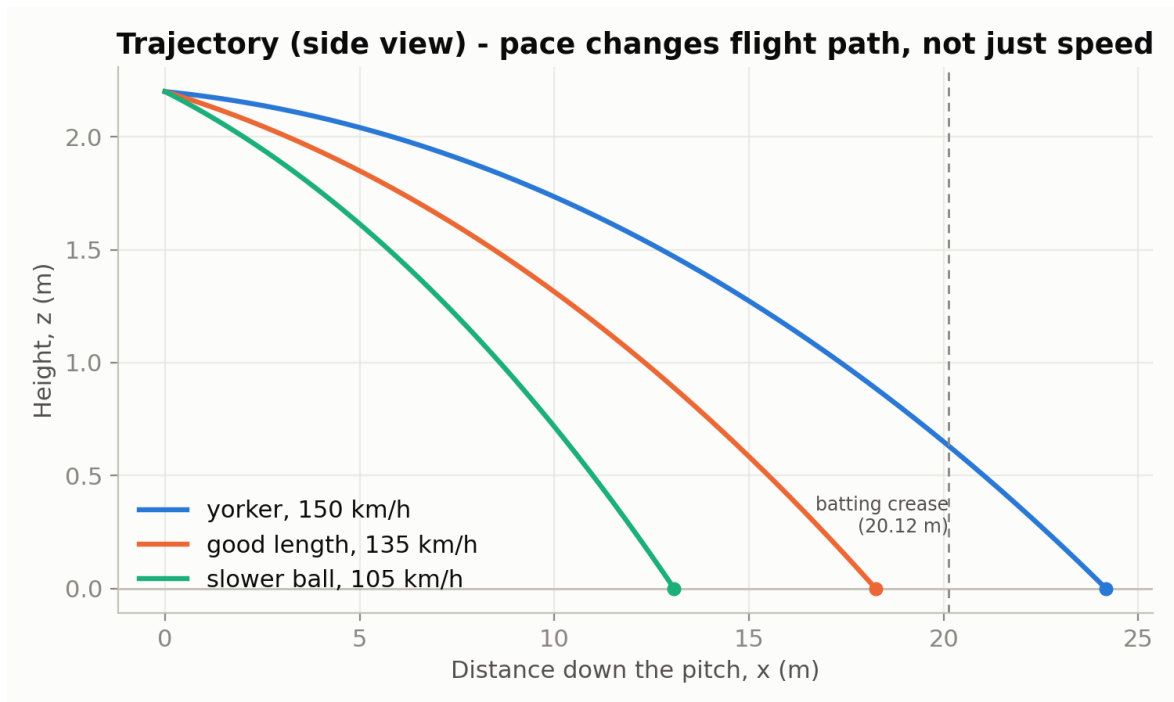


Figure 7. Simulated side-on trajectories at three release speeds, same release height and similar launch angle. Faster deliveries aren't just "the same parabola, sooner" — drag reshapes the curve itself, and this is why a genuine yorker's late dip surprises batters: the trajectory is measurably flatter for longer, then drops away faster than a slow-ball parabola would predict.

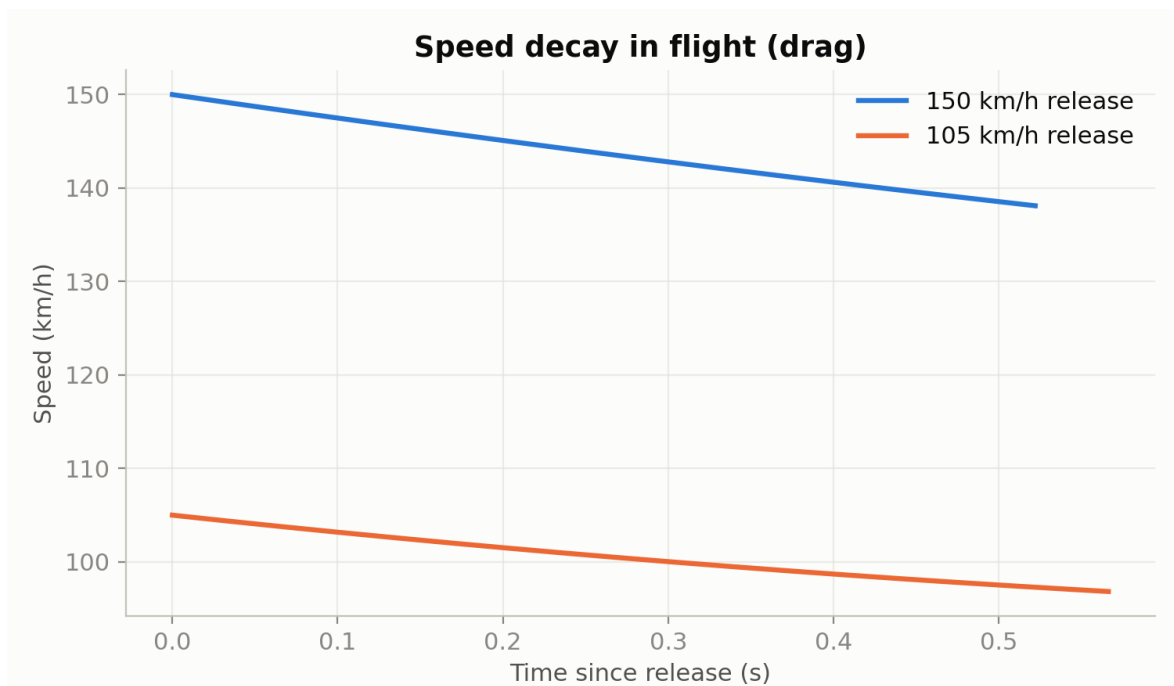


Figure 8. Simulated speed loss through the air. Both deliveries lose roughly 8-10% of their release pace by the time they reach the pitch — a bowling machine's stated "release speed" and a radar gun reading taken near the batter will not match, and this is why.

5. Worked results

Numbers from the reference simulator (Section 6) for a representative set of deliveries, all released from 2.2 m (2.0 m for the spin deliveries) with default ball and air parameters (Section 8).

Delivery	Flight (s)	Pitches at (m)	Lateral dev. (cm)	Speed lost (%)
Yorker, 150 km/h, seam-up	0.61	24.17	+0.0	9.5
Outswinger, 135 km/h	0.51	18.22	+78.8	8.1
Inswinger, 135 km/h	0.51	18.22	-78.8	8.1
Reverse-swing yorker, 150 km/h	0.58	22.90	+124.7	9.1
Leg-break, 85 km/h, 1800 rpm	0.74	16.38	-46.6	8.0
Slower ball, 105 km/h, seam-up	0.44	12.27	+0.0	5.9

"Pitches at" is distance from release along x ; the batting crease sits at $x = 20.12$ m. Lateral deviation is the perpendicular distance between the pitching point and a straight line drawn from release along the initial aim direction — positive is off-side drift for a right-handed batter under this model's sign convention.

6. The complete algorithm

Two algorithms, both operating on the same physics: a forward simulator (release conditions → trajectory) and, in Section 7, an inverse mapping (desired delivery → machine wheel speeds).

6.1 Forward trajectory simulation

```
FUNCTION acceleration(state, ball, env, delivery):
    x, y, z, vx, vy, vz = state
    rv = (vx, vy, vz) - env.wind                # velocity relative to air
    speed = |rv|
    a = (0, 0, -g)                               # gravity

    # --- drag ---
    Re = env.rho * speed * ball.diameter / env.mu
    Cd = drag_coefficient(Re)                    # logistic drag-crisis curve
    a += -0.5 * env.rho * Cd * ball.area * speed * rv / ball.mass

    # --- Magnus (spin) ---
    IF delivery.spin_rate > 0:
        Sp = ball.radius * delivery.spin_rate / speed
        Cl = magnus_lift_coefficient(Sp)         # saturating curve
        dir = normalize(delivery.spin_axis) x normalize(rv)
        a += 0.5 * env.rho * Cl * ball.area * speed^2 * dir / ball.mass

    # --- swing (seam) ---
    IF delivery.seam_angle > 0:
        Cs = swing_coefficient(delivery.seam_angle, speed,
                                delivery.reverse_swing, reverse_threshold)
        lat = horizontal_perp(rv) * delivery.swing_sign
        a += 0.5 * env.rho * Cs * ball.area * speed^2 * lat / ball.mass

        # companion effect: the same seam presentation also costs extra
        # drag, on top of the Re-based Cd above -- see Section 3.5
        Cds = seam_drag_coefficient(delivery.seam_angle)
        a += -0.5 * env.rho * Cds * ball.area * speed * rv / ball.mass

    RETURN a

FUNCTION simulate(ball, env, delivery):
    state = delivery.initial_state()             # release pos + velocity
    t = 0
    WHILE state.z > 0:
        a = acceleration(state, ball, env, delivery)
        # adaptive RK45 step (solve_ivp) -- shown here as simple Euler
        state.velocity += a * dt
        state.position += state.velocity * dt
        t += dt
        RECORD(t, state)
    RETURN trajectory
```

6.2 Reading the results

From the recorded trajectory: flight time is the final t ; the pitching point is (x, y) at the last recorded state; lateral deviation compares the final y against a straight line from the release aim direction; speed lost compares initial and final $|v|$. All four are the columns in the Section 5 table.

7. From physics to a bowling machine

Everything above models the ball once it has already left the machine. A physical bowling machine has no wrist or fingers — a twin-wheel design compresses the ball briefly between two independently-driven wheels and flings it out. The wheels' surface speeds set both the ball's release speed and its spin:

$$V_{ball} = \eta_v \cdot \frac{V_1 + V_2}{2}$$

$$\omega_{ball} = \eta_\omega \cdot \frac{V_1 - V_2}{2r}$$

v_1 and v_2 are the two wheels' surface (tangential) speeds, r is the ball radius, and η_v , η_ω are slip/efficiency factors below 1 — real contact between a rubber wheel and a leather ball always loses some speed and spin to slip, and spin transfers less efficiently than forward drive. Their **average** surface speed drives the ball forward; their **difference** imparts spin, exactly like rolling a pen between your palms moving at different speeds.

Inverting these two equations gives the control mapping you actually need — the wheel speeds required to produce a chosen delivery:

```
FUNCTION wheel_rpms_for_delivery(ball_speed, spin_rate, ball, machine):
    sum_v = 2 * ball_speed / machine.speed_efficiency
    diff_v = 2 * ball.radius * spin_rate / machine.spin_efficiency
    v1 = (sum_v + diff_v) / 2
    v2 = (sum_v - diff_v) / 2
    rpm1 = v1 * 60 / (2 * pi * machine.wheel_radius)
    rpm2 = v2 * 60 / (2 * pi * machine.wheel_radius)
    RETURN rpm1, rpm2
```

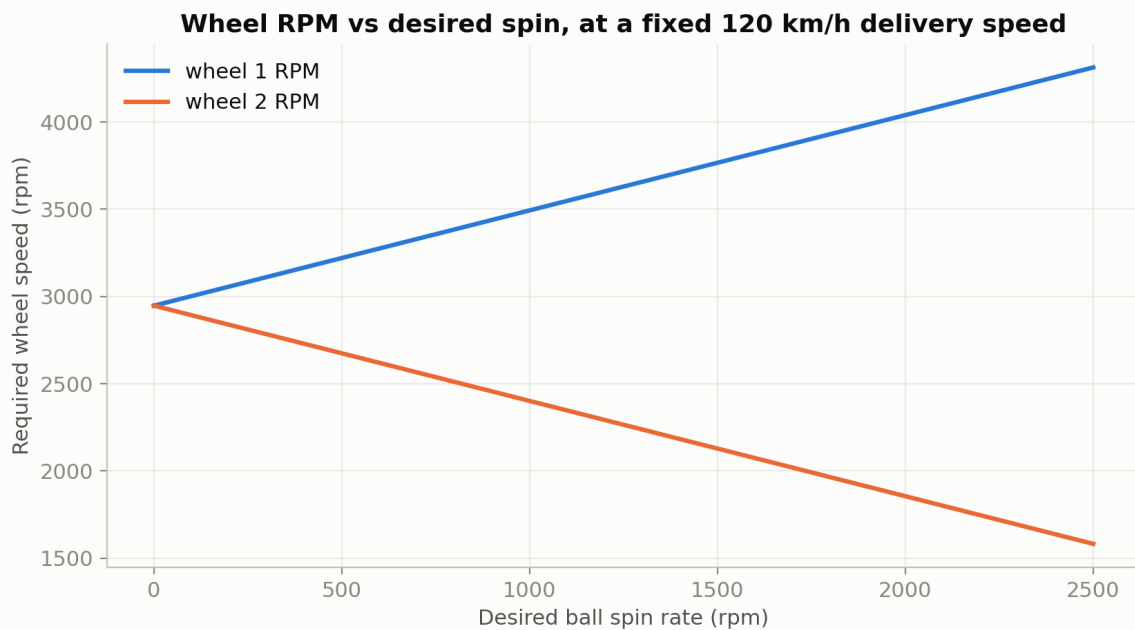


Figure 9. Required wheel RPM for a fixed 120 km/h delivery as desired spin rate increases, with a 0.12 m wheel radius, 90% speed efficiency and 55% spin efficiency. At zero spin both wheels turn at the same speed; as spin rate rises one wheel speeds up and the other slows, with the gap between them setting the spin.

7.1 Setting the spin axis and seam

The wheel pair's angular velocity difference imparts spin about an axis in the plane perpendicular to the line joining the two wheels. Rotating the whole wheel-pair assembly about the barrel axis rotates that spin axis: roughly 0° gives a vertical spin axis (off-break/leg-break depending on which wheel is faster), roughly 90° gives a horizontal spin axis (top-spin/back-spin). Seam angle is a separate, largely mechanical parameter on most twin-wheel machines — set by how the ball is oriented when it is fed in — rather than something the wheel speeds control directly; if your machine has a seam-orientation feeder, treat its setting as the delivery's `seam_angle_deg` input to Section 6.1 directly.

8. Calibration: making this predictive, not just illustrative

Every coefficient in this document is a reasonable default, not a measurement of your specific machine and ball. Three calibration steps turn it from illustrative to predictive:

- **Speed efficiency (η_v):** set both wheels to equal RPM, measure exit speed with a radar gun, and solve $\eta_v = v_{\text{ball}} / v_{\text{wheel}}$ at a few different wheel speeds — it may not be perfectly constant across the speed range.
- **Spin efficiency (η_ω):** fix a wheel-speed differential, film the ball leaving the machine at high frame rate (or with a strobe), count revolutions over a known distance to get actual spin rate, and solve η_ω from the wheel-spin equation.
- **Aerodynamic coefficients (C_d , C_l , C_s):** track real deliveries (phone slow-motion with visible pitch markings, or a tracking camera if available), fit the drag-crisis curve's Re_{crit} and width, the Magnus curve's $C_{l,\text{max}}$, and the swing curve's peak angle and $C_{s,\text{max}}$ against measured pitching points and flight times, e.g. with a least-squares fit.

9. Parameter reference

Parameter	Symbol	Default	Units
Ball mass	m	0.160	kg
Ball radius	r	0.0360	m
Air density (sea level)	ρ	1.225	kg/m ³
Air dynamic viscosity	μ	1.81e-5	Pa·s
Gravity	g	9.81	m/s ²
Subcritical drag coeff.	$C_{d,sub}$	0.50	-
Supercritical drag coeff.	$C_{d,sup}$	0.22	-
Critical Reynolds number	Re_{crit}	1.7e5	-
Drag-crisis transition width	w	2.5e4	-
Max Magnus lift coefficient	$C_{l,max}$	0.35	-
Max swing coefficient	$C_{s,max}$	0.30	-
Swing peak-shape exponent	n	6	-
Resulting peak swing seam angle	-	~22	degrees
Max extra seam-drag coefficient	$C_{ds,max}$	0.06	-
Reverse-swing pace threshold	-	140	km/h
Wheel radius (machine)	R_w	0.12	m
Speed transfer efficiency	η_v	0.90	-
Spin transfer efficiency	η_ω	0.55	-
Pitch length	-	20.12	m

All defaults are the values used to generate every figure and table in this document, and match the reference implementation in Appendix A.

10. Limitations and honest caveats

- Coefficients are illustrative empirical approximations, not measurements of a specific ball — see Section 8 before relying on absolute numbers.
- No CFD or turbulence modelling of any kind (RANS with a $k\text{-}\epsilon/k\text{-}\omega$ closure, LES, DNS) was performed anywhere in this document. The drag, Magnus, swing and seam-drag coefficients are curve-fits standing in for that physics, not results derived from simulating the airflow — the boundary-layer transition described in Section 3.2 is the explanation for the drag-crisis curve's shape, not something the algorithm computes.
- Spin decay in flight (air resistance slowing the ball's rotation) is ignored; over a bowling-length flight it is typically a small effect, but it is not zero.

- Swing is modelled as a constant-direction force fixed at release; real seam wobble changes swing unpredictably through the flight, which this model does not capture.
- Reverse swing is a simplified toggle, not a first-principles prediction — the underlying physics is still actively researched.
- The $\cos(\theta)/\sin(\theta)$ split between swing and extra seam drag (Sections 3.4-3.5) is a geometrically-motivated resolution, not two independently wind-tunnel-measured curves — the peak-shape exponent n and $C_{ds,max}$ are still fitted constants, best replaced with your own measurements per Section 8.
- The machine release model assumes an idealised twin-wheel geometry with no slip beyond the single efficiency factors η_v and η_ω ; a three-wheel machine, or one with a very different contact geometry, will need its own version of Section 7.
- Air density is a single configurable constant per run (`Environment.at_altitude` gives a rough correction); temperature, humidity and wind are not modelled beyond a constant wind vector.

Appendix A — Reference implementation (Python)

The core of the forward simulator, in full — this is the actual code the figures and tables in this document were generated from, included so it can be copied directly into a project.

aerodynamics.py — coefficient models

```
"""
Empirical aerodynamic coefficient models.

None of these are exact -- real values come from wind-tunnel and field-tracking
studies (e.g. Mehta's work on cricket ball aerodynamics) and vary with surface
condition, humidity and individual balls. They're written as small, isolated
functions so you can swap in a better-calibrated curve (e.g. fitted to Hawk-Eye
trajectory data) without touching the integrator.
"""

from __future__ import annotations

import math

def reynolds_number(speed_mps: float, diameter_m: float, air_density: float, viscosity: float) -> float:
    return air_density * speed_mps * diameter_m / viscosity

def drag_coefficient(reynolds: float,
                    cd_subcritical: float = 0.5,
                    cd_supercritical: float = 0.22,
                    re_critical: float = 1.7e5,
                    re_width: float = 2.5e4) -> float:
    """
    Smooth logistic approximation of the cricket-ball 'drag crisis': at low
    Reynolds number the boundary layer separates early (laminar), giving high
    drag; past a critical Re the layer trips turbulent and stays attached
    longer, so drag drops sharply. Bowling speeds (Re ~ 1e5-2e5) sit right in
    this transition, which is part of why swing bowling is even possible.
    """
    if reynolds <= 0:
        return cd_subcritical
    x = (reynolds - re_critical) / re_width
    # sigmoid from 1 (low Re) to 0 (high Re)
    sigmoid = 1.0 / (1.0 + math.exp(x))
    return cd_supercritical + (cd_subcritical - cd_supercritical) * sigmoid

def magnus_lift_coefficient(spin_parameter: float, cl_max: float = 0.35) -> float:
    """
    Lift coefficient for a spinning sphere as a function of the dimensionless
    spin parameter  $Sp = r \cdot \omega / v$ . Uses the common saturating approximation
     $Cl = Sp / (a + b \cdot Sp)$ , which rises roughly linearly for small  $Sp$  (gentle
    spin) and flattens out at high  $Sp$  (fast spin relative to ball speed) --
    consistent with measurements on spinning balls in several sports.
    """
    if spin_parameter <= 0:
        return 0.0
    a, b = 0.5, 1.0 / cl_max - 0.5 if cl_max else 1.0
    # simplifies to a saturating curve that -> cl_max as spin_parameter -> inf
    return spin_parameter / (a + spin_parameter / cl_max)

def _seam_force_peak(peak_exponent: float) -> float:
    """Peak value of  $\sin(\theta) \cdot \cos(\theta)^n$  over  $\theta$  in  $[0, 90\text{deg}]$ , used to
    normalize the seam-force projections below so their maximum equals the
    requested coefficient exactly at the resulting peak angle."""
```

```

theta_peak = math.atan(1.0 / math.sqrt(peak_exponent))
return math.sin(theta_peak) * math.cos(theta_peak) ** peak_exponent

def swing_coefficient(seam_angle_deg: float,
                    speed_mps: float,
                    reverse_swing: bool,
                    reverse_threshold_mps: float,
                    cs_max: float = 0.30,
                    reverse_gain: float = 1.15,
                    peak_exponent: float = 6.0) -> float:
    """
    Magnitude of the seam-swing (lateral) force coefficient, derived from a
    force-resolution argument rather than fitted directly as a lateral force:
    the seam generates some boundary-layer-driven side force, and geometry
    says the component of that force perpendicular to the direction of
    travel (the swing itself) should carry a cos(theta) projection -- as the
    seam plane rotates away from being broadside to the flow, less of the
    asymmetry's force acts sideways and more of it acts along the direction
    of travel. Combined with the force's own growth from zero frontal
    exposure at theta=0 (a sin(theta) factor), the natural form is:

        C_s(theta) = A0 * sin(theta) * cos(theta)^n

    which is zero at 0 degrees (no asymmetry to project) and at 90 degrees
    (fully side-on, all of it projects into drag instead -- see
    seam_drag_coefficient() below for that companion effect). A plain
    sin(theta)*cos(theta) (n=1) would peak at 45 degrees; the exponent n
    (default 6) folds in the empirically-observed fact that the seam's own
    boundary-layer disruption saturates well before that, pulling the
    combined peak down to the classically-quoted "ideal" seam angle of ~22
    degrees. A0 is solved so the peak value equals cs_max exactly.

    If reverse_swing is enabled and the ball is travelling above the reverse
    threshold, the effective coefficient is boosted (roughness-driven
    transition dominates over the conventional seam effect at high pace) --
    this is a simplified stand-in for a genuinely complex, still-researched
    phenomenon; treat it as illustrative rather than predictive.
    """
    theta = math.radians(min(max(seam_angle_deg, 0.0), 90.0))
    a0 = cs_max / _seam_force_peak(peak_exponent)
    base = a0 * math.sin(theta) * math.cos(theta) ** peak_exponent

    if reverse_swing and speed_mps > reverse_threshold_mps:
        return base * reverse_gain
    return base

def seam_drag_coefficient(seam_angle_deg: float, cds_max: float = 0.06) -> float:
    """
    Extra drag from presenting the seam at an angle to the airflow, on top of
    drag_coefficient()'s Reynolds-based value.

    This is deliberately a SEPARATE curve from swing_coefficient(), not a
    second projection of the same force -- the two are different physical
    mechanisms in real cricket-ball aerodynamics. Orthodox swing comes from a
    boundary-layer transition asymmetry that peaks at a moderate seam angle
    (~20-25 degrees) and is genuinely small again by 90 degrees. Extra seam
    drag comes from simple frontal blockage -- how much of the raised seam's
    profile faces the oncoming air -- which keeps growing all the way to a
    seam fully side-on. That's exactly the "cross-seam" ball bowlers use on
    purpose: seam at ~90 degrees, no swing at all, but a slower, more
    turbulent, harder-gripping delivery off the pitch. Modelled simply as
    C_ds(theta) = C_ds,max * sin(theta)^2 -- zero with the seam aligned dead
    straight, maximum fully side-on.
    """
    theta = math.radians(min(max(seam_angle_deg, 0.0), 90.0))
    return cds_max * math.sin(theta) ** 2

```

dynamics.py — force summation

```
"""
The force-summing function: given the ball's current state, compute total
acceleration. This is the single function the integrator calls every step.
"""

from __future__ import annotations

import math
from typing import Sequence

from .aerodynamics import (
    reynolds_number,
    drag_coefficient,
    magnus_lift_coefficient,
    swing_coefficient,
    seam_drag_coefficient,
)
from .ball import BallProperties, Environment, Delivery

def _cross(a, b):
    return (
        a[1] * b[2] - a[2] * b[1],
        a[2] * b[0] - a[0] * b[2],
        a[0] * b[1] - a[1] * b[0],
    )

def _norm(v):
    return math.sqrt(v[0] ** 2 + v[1] ** 2 + v[2] ** 2)

def acceleration(state: Sequence[float],
                 ball: BallProperties,
                 env: Environment,
                 delivery: Delivery) -> tuple:
    """
    state = (x, y, z, vx, vy, vz). Returns (ax, ay, az) -- the sum of gravity,
    drag, Magnus (spin) and swing (seam), divided by mass.
    """
    _, _, _, vx, vy, vz = state
    wx, wy, wz = env.wind_mps

    # velocity relative to the air the ball is actually flying through
    rvx, rvy, rvz = vx - wx, vy - wy, vz - wz
    speed = _norm((rvx, rvy, rvz))

    m = ball.mass_kg
    A = ball.area_m2
    rho = env.air_density_kg_m3
    diameter = 2 * ball.radius_m

    # --- gravity -----
    ax, ay, az = 0.0, 0.0, -env.gravity_mps2

    if speed < 1e-6:
        return ax, ay, az # ball at rest relative to air; only gravity acts

    # --- drag -----
    Re = reynolds_number(speed, diameter, rho, env.dynamic_viscosity_pa_s)
    Cd = drag_coefficient(Re)
    drag_scalar = -0.5 * rho * Cd * A * speed / m
    ax += drag_scalar * rvx
    ay += drag_scalar * rvy
    az += drag_scalar * rvz
```

```

# --- Magnus (spin) force -----
omega = delivery.spin_rad_s
spin_rate = _norm(omega)
if spin_rate > 1e-6:
    spin_parameter = ball.radius_m * spin_rate / speed
    Cl = magnus_lift_coefficient(spin_parameter)
    omega_hat = tuple(o / spin_rate for o in omega)
    v_hat = tuple(v / speed for v in (rvx, rvy, rvz))
    direction = _cross(omega_hat, v_hat) # already unit-ish, cross of two unit vectors
    magnus_scalar = 0.5 * rho * Cl * A * speed ** 2 / m
    ax += magnus_scalar * direction[0]
    ay += magnus_scalar * direction[1]
    az += magnus_scalar * direction[2]

# --- swing (seam) force -----
if delivery.seam_angle_deg > 1e-6:
    Cs = swing_coefficient(
        seam_angle_deg=delivery.seam_angle_deg,
        speed_mps=speed,
        reverse_swing=delivery.reverse_swing,
        reverse_threshold_mps=delivery.reverse_threshold_kmh / 3.6,
    )
    # Lateral direction: horizontal, perpendicular to the ball's initial
    # direction of travel (seam held roughly fixed relative to the seam
    # bowler set at release, not tied to the ball's spin axis).
    fwd = (rvx, rvy, 0.0)
    fwd_norm = _norm(fwd) or 1.0
    fwd_hat = (fwd[0] / fwd_norm, fwd[1] / fwd_norm, 0.0)
    lateral_hat = (-fwd_hat[1], fwd_hat[0], 0.0) # rotate 90 deg in the horizontal plane
    swing_scalar = 0.5 * rho * Cs * A * speed ** 2 / m * delivery.swing_sign
    ax += swing_scalar * lateral_hat[0]
    ay += swing_scalar * lateral_hat[1]
    az += swing_scalar * lateral_hat[2]

    # Companion component of the SAME seam-generated force, resolved
    # along the direction of travel instead of across it: presenting the
    # seam at an angle doesn't just deflect the ball, it also costs a
    # little extra drag on top of drag_coefficient()'s Re-based value.
    # This opposes velocity exactly like the main drag term above.
    Cds = seam_drag_coefficient(seam_angle_deg=delivery.seam_angle_deg)
    seam_drag_scalar = -0.5 * rho * Cds * A * speed / m
    ax += seam_drag_scalar * rvx
    ay += seam_drag_scalar * rvy
    az += seam_drag_scalar * rvz

return ax, ay, az

def state_derivative(t: float, state: Sequence[float],
                    ball: BallProperties, env: Environment, delivery: Delivery):
    """d/dt of (x, y, z, vx, vy, vz) -- the function solve_ivp integrates."""
    x, y, z, vx, vy, vz = state
    ax, ay, az = acceleration(state, ball, env, delivery)
    return [vx, vy, vz, ax, ay, az]

```

machine.py — wheel-speed control mapping

```
"""
Release-mechanism model for a twin-wheel bowling machine.

Everything in dynamics.py / simulate.py models the ball once it's already in
the air. This module models the other end of the problem: a bowling machine
doesn't have a wrist and fingers, it has two (sometimes three) motor-driven
wheels that compress the ball between them and fling it out. This maps
between "what the wheels are doing" and "what the ball does at release" --
i.e. the actual control variables you'd send to motor controllers.

Physical picture (twin-wheel machine):
    Two wheels of radius R_w, spinning with surface (tangential) speeds
    v1 and v2, squeeze the ball briefly as it passes between them.
    - Their AVERAGE surface speed sets the ball's exit speed (both wheels
      drive the ball forward together).
    - Their DIFFERENCE sets the ball's spin (one wheel's surface moves
      faster than the other across the contact patch, which torques the
      ball -- exactly like rolling a pen between your palms at different
      speeds).
    An efficiency/slip factor < 1 is included because real contact always
    slips a little; calibrate it against your own machine with a radar gun
    and a slow-motion spin count, then treat it as fixed for that machine.
"""

from __future__ import annotations

import math
from dataclasses import dataclass

from .ball import BallProperties

@dataclass
class WheelMachine:
    wheel_radius_m: float = 0.12      # typical drive-wheel radius on a compact machine
    speed_efficiency: float = 0.90     # slip factor for forward speed transfer
    spin_efficiency: float = 0.55      # slip factor for spin transfer (spin slips more than drive)

    # --- forward model: wheel speeds -> ball release state -----

    def wheel_rpm_to_surface_speed(self, rpm: float) -> float:
        return 2 * math.pi * self.wheel_radius_m * rpm / 60.0

    def ball_release_state(self, rpm1: float, rpm2: float, ball: BallProperties):
        """Given both wheel RPMs, return (ball_speed_mps, spin_rate_rad_s)."""
        v1 = self.wheel_rpm_to_surface_speed(rpm1)
        v2 = self.wheel_rpm_to_surface_speed(rpm2)
        ball_speed = self.speed_efficiency * (v1 + v2) / 2.0
        spin_rate = self.spin_efficiency * (v1 - v2) / (2.0 * ball.radius_m)
        return ball_speed, spin_rate

    # --- inverse model: desired ball release state -> wheel speeds -----

    def wheel_rpms_for_delivery(self, ball_speed_mps: float, spin_rate_rad_s: float,
                                ball: BallProperties):
        """
        Solve the forward equations backwards:
            v1 + v2 = 2 * ball_speed / speed_efficiency
            v1 - v2 = 2 * radius * spin_rate / spin_efficiency
        for v1, v2, then convert each to a wheel RPM.
        """
        sum_v = 2 * ball_speed_mps / self.speed_efficiency
        diff_v = 2 * ball.radius_m * spin_rate_rad_s / self.spin_efficiency
        v1 = (sum_v + diff_v) / 2.0
        v2 = (sum_v - diff_v) / 2.0
        rpm1 = v1 * 60.0 / (2 * math.pi * self.wheel_radius_m)
```

```

rpm2 = v2 * 60.0 / (2 * math.pi * self.wheel_radius_m)
return rpm1, rpm2

def wheel_pair_tilt_for_spin_axis(self, spin_type: str) -> float:
    """
    Rotation of the wheel-pair assembly about the barrel's own axis (deg),
    needed to align the spin axis with a given stock delivery. The wheel
    pair imparts spin about an axis in the plane perpendicular to the
    line joining the two wheels -- rotating that plane rotates the axis:
        0 deg -> side-spin axis vertical (z) : off/leg break
        90 deg -> side-spin axis horizontal (y) : top/backspin
    Intermediate angles blend the two (useful for a "spinning seam" ball
    that both drifts and dips).
    """
    table = {
        "backspin": 90.0, "topspin": 90.0,
        "offspin": 0.0, "legspin": 0.0,
        "none": 0.0,
    }
    return table.get(spin_type, 0.0)

```